FHE’de Kuantum Güvenliği Yetkilendirmede Biter

Şifre çözme yetkisinin yaşam döngüsü için asgari bir tasarım: Emekli Et ve Taşı

Semih Civelek (Himess)

21 Ekim 2026 · Taslak v0.1 · Yayın öncesi Zama ile paylaşılacaktır

İçindekiler

Özet
1. Giriş
2. Arka plan: fhEVM’de şifre çözme nasıl çalışır?
3. Tehdit modeli
4. Bulgular
5. Tasarım ilkeleri
6. Değerlendirilen ve reddedilen alternatifler
7. Önerilen tasarım: Emekli Et ve Taşı
8. Güvenlik analizi
9. Maliyet
10. Değerlendirme planı
11. Sınırlamalar ve açık sorular
12. İlgili çalışmalar
13. Sorumlu paylaşım
Ek A. Kod referansları (20 Ekim 2026, `main`)

Özet

Zincir üstü FHE sistemleri (Zama fhEVM, Fhenix CoFHE) kendilerini “kuantuma dayanıklı” olarak tanıtır. Şifreli verinin kendisi için bu doğrudur. TFHE şifreli metinleri kafes (LWE) tabanlıdır, Zama’nın kullanıcıya şifre çözme kanalı da ML-KEM kullanır. Ancak bu sistemlerde **bir verinin kimin için çözüleceğine karar veren katman** tamamen ECDSA’ya dayanır. Şifreli verinin güvenliği, bu yetkilendirme katmanının güvenliğinden fazla olamaz.

Zama’nın 20 Ekim 2026 tarihli açık kaynak kodunu satır satır inceledik ve şu bulgulara ulaştık:

- Bir hesabın şifre çözme hakkı adresine **kalıcı** olarak bağlıdır ve kullanıcı bu hakkı geri alamaz.

- İmza iptal mekanizması sızan *imzalara* karşı tasarlanmıştır. Sızan *anahtara* karşı bir şey yapmaz.
- Anahtarı ele geçiren biri, hesabın geçmişteki bütün şifreli verilerini tek işlemle başka bir adrese aktarabilir ya da herkese açık hale getirebilir.
- Şifre çözme izinleri zincir dışında `ecrecover` ile doğrulanır. Bu yüzden ne Ethereum'un kuantum acil durum planı (EOA dondurma) ne de EOA anahtarını devre dışı bırakan EIP'ler (7851, 8151) bu kapıyı kapatır.
- Akıllı hesap desteği (ERC-1271) vardır, ama 250.000 gas'lık doğrulama tavanı bugünkü bütün PQ imza doğrulayıcılarını dışarıda bırakır.

Fhenix CoFHE'de de aynı sınıftan sorunlar vardır.

Bu sorunlar yalnızca kuantum gelecekte değil, **bugün** de geçerlidir: anahtarı phishing ile çalınan bir kullanıcının gizli geçmişi kalıcı olarak açığa çıkar. Bir kuantum bilgisayar ise aynı sonucu, açık anahtarı görünmüş her hesap için aynı anda üretir.

Bu çalışma yeni bir kriptografik yapı önermiyor. Önerdiği şey, mevcut mekanizmaların (hesap soyutlama, ACL, imza iptali) eksik kalan beş küçük parçasıdır. Tasarıma **"Emekli Et ve Taşı"** (Retire & Carry) adını verdik. Bu parçalarla bir kullanıcı:

1. geçmişini kuantuma dayanıklı bir akıllı hesaba taşıyabilir,
2. eski ECDSA anahtarını şifre çözme açısından geri dönülmez biçimde emekliye ayırabilir,
3. protokol de kuantum günü ilan edildiğinde "kapalı kalarak" (fail-closed) geçmişi korur.

Önerinin büyük kısmı geriye dönük uyumludur ve var olan kullanıcıları etkilemez. Bölüm 10'da Sepolia ve yerel test ortamında yapılacak doğrulama deneyleri tanımlanmıştır.

1. Giriş

1.1 Problem

Kuantum sonrası gizlilik tartışması bugüne kadar büyük ölçüde **zincirde duran şifreli notlar** üzerine yoğunlaştı. Konu şuydu: ECDH ile şifrelenmiş not, gelecekte bir kuantum bilgisayarla çözülebilir ("şimdi topla, sonra çöz", HNDL). FHE tarafında ise genel kanı "FHE zaten kafes tabanlı, dolayısıyla kuantuma dayanıklı" yönünde. Örneğin 17 Ekim 2026'da yayılan bir Fhenix tanıtım içeriği bu iddiayı hiçbir uyarı eklemeden yapıyor [R13].

Bu çalışmanın tezi şu: **FHE sistemlerinde gizlilik, şifrenin gücüyle değil, şifre çözme oracle'ının kime "evet" dediğiyle belirlenir.** Zama'nın kendi belgeleri de KMS'in gRPC şifre çözme ucunun kendi başına erişim kontrolü yapmadığını, isteklerin Gateway tarafından doğrulanması gerektiğini söylüyor [R1]. Bu doğrulamanın tamamı ECDSA'ya dayandığı için, ECDSA'nın düştüğü

gün (klasik bir anahtar hırsızlığıyla tek bir kullanıcı için ya da bir kuantum bilgisayarla herkes için) şifrenin kafes tabanlı olması hiçbir şeyi korumaz. Saldırganın şifreyi kırmasına gerek kalmaz. Meşru sahibin imzasını üretir, oracle da veriyi ona çözer.

Bu tehdide **oracle-retroaktif maruziyet** diyoruz. Klasik HNDL'den farkı şudur: Saldırganın önceden bir şey toplamasına gerek yoktur, çünkü veri zaten zincirde ve oracle hâlâ çalışıyor. Şifreyi kırmasına da gerek yoktur, çünkü yetkiyi taklit etmesi yeterlidir.

1.2 Katkılar

1. **Tehdit modeli:** Şifre çözme oracle'ına dayanan gizlilik sistemleri (FHE, eşik şifreleme, TEE) için yetkilendirme odaklı bir kuantum tehdit modeli (Bölüm 3).
2. **Kod düzeyinde bulgular:** Zama fhEVM'de yedi bulgu (B1–B7). Her biri için "bilinçli tasarım tercihi mi, gözden kaçan bir durum mu" ayrımı yapıldı. Fhenix CoFHE ile karşılaştırma da eklendi (Bölüm 4).
3. **Asgari tasarım:** "Emekli Et ve Taşı" — beş bileşen. Değerlendirdiğimiz ve reddettiğimiz altı alternatif de gerekçeleriyle birlikte veriliyor; reddedilenlerden biri bizim ilk önerimiz (Bölüm 6–7).
4. **Değerlendirme planı:** Sepolia ve yerel fork üzerinde tekrarlanabilir deneyler (Bölüm 10).

1.3 Yöntem

Aşağıdaki repoların 20 Ekim 2026 tarihli `main` dallarını klonladık; commit geçmişlerini, uzak dalları ve kaynak kodu okuduk:

- `zama-ai/fhevm` @ `ed85171`
- `zama-ai/kms` @ `c755e84`
- `zama-ai/tfhe-rs` @ `0a43e3c`
- `FhenixProtocol/cofhe-contracts` @ `b5c20ed`

Kamuya açık forum ve standart tartışmalarını da tarayıp ilgili ERC/EIP'leri inceledik. **Önemli uyarı:** `main` dalı, mainnet ya da Sepolia'da çalışan sürümle aynı olmayabilir. Bölüm 10'daki ilk deneyin amacı da bunu doğrulamaktır.

2. Arka plan: fhEVM'de şifre çözme nasıl çalışır?

- **Handle:** Zincirde tutulan, şifreli bir değere işaret eden 32 baytlık kimlik. Gerçek TFHE şifreli metni coprocessor'larda saklanır.
- **ACL (host zincirinde):** Hangi adresin hangi handle'ı kullanabileceğini tutar.
  - `allow(handle, account)`: kalıcı izin verir.
  - `allowTransient`: yalnızca o işlem süresince geçerli izin verir.
  - `allowForDecryption(handles)`: handle'ı "herkese açık çözülebilir" yapar.
  - Kullanıcı şifre çözmeyi delegeye devredebilir: `delegateForUserDecryption`.

    - `invalidateDecryptionSignaturesBefore`: kullanıcının eski imzalarını iptal etmesini sağlar. [R2]
- **Kullanıcı şifre çözmesi (user decryption):**
    1. Kullanıcı bir EIP-712 izni imzalar. İzin şunları içerir: kendi adresi, tek seferlik bir ML-KEM açık anahtarı, izin verilen sözleşmeler ve geçerlilik penceresi.
    2. Relayer bu izni KMS connector'a iletir.
    3. Connector, host zincirinden okuma yaparak üç kontrol yapar: imza (`ecrecover`, olmazsa ERC-1271), iptal zaman damgası ve her handle için ACL [R3].
    4. KMS düğümleri eşik MPC ile çözüm paylarını ML-KEM-512 + AES-256-GCM ile kullanıcının anahtarına şifreleyip ECDSA ile imzalar.
- **Herkese açık şifre çözme (public decryption):** `allowForDecryption` ile işaretlenen handle'ı herkes çözdürebilir. KMS sonucu ECDSA ile imzalar ve sonuç zincirde `KMSVerifier` ile doğrulanır.
- **ERC-7984:** OpenZeppelin ve Zama'nın gizli token standardı. Bakiyeler ve transfer tutarları handle'lardır; token sözleşmesi bunları ilgili hesaplara `allow` eder. Bir hesabın "gizli geçmişi", ona izin verilmiş bütün handle'lardan oluşur.

3. Tehdit modeli

Korunan varlıklar:

- V1: Bir hesaba izin verilmiş geçmiş handle'ların düz metinleri (bakiyeler, transfer tutarları, açık artırma teklifleri…).
- V2: Gelecekte oluşacak handle'ların düz metinleri.

Saldırganlar:

Kod	Saldırgan	Yetenek	Zaman
T1	Klasik anahtar hırsızı	Tek bir kullanıcının ECDSA özel anahtarını ele geçirir (phishing, kötü amaçlı yazılım, sızıntı)	Bugün
T2	Kuantum saldırgan (CRQC)	Açık anahtarı zincirde görünmüş her secp256k1 anahtarını türetir	Q-günü ve sonrası

Varsayımlar: KMS eşik güvenliği (13 düğümden en az 2/3'ü dürüst), TFHE ve ML-KEM'in güvenliği. KMS'in kendi imzalarının PQ geçişi Zama'nın yol haritasındadır (bkz. B7) ve kapsam dışıdır.

Hedefler:

- G1: Kullanıcı, ele geçirilmiş ya da ileride ele geçirilecek bir ECDSA anahtarının V1'e erişimini **önceden** kalıcı olarak kesebilmeli.
- G2: Kullanıcı, geçmişini kaybetmeden PQ bir hesaba geçebilmeli.
- G3: PQ hesaplar şifre çözmeyi makul maliyetle yetkilendirebilmeli.
- G4: Q-günü ilan edildiğinde protokol, ECDSA yetkilerini reddederek "kapalı kalabilmeli".
- G5: Var olan kullanıcı ve uygulamalar için geriye dönük uyumluluk.
- G6: Asgari değişiklik. Yeni kriptografik primitif yok, yeni güven varsayımı yok.

Kapsam dışı: Kullanıcının harekete geçmeden önce çalınan anahtarı. Hiçbir tasarım geçmişe dönük hırsızlığı geri alamaz. Ayrıca karşı tarafların (alıcılar, sözleşmeler) aynı handle'lar üzerindeki kendi izinleri; onları ilgili hesapların sahipleri korumalıdır.

4. Bulgular

Her bulgu için kod referansı ve "bilinçli tercih mi, gözden kaçan mı" değerlendirmesi verilmiştir. Kod referanslarının tamamı Ek A'dadır.

B1. Şifre çözme hakkı adrese kalıcı bağlıdır; kullanıcı geri alamaz

`allow` kalıcı bir eşleme yazar: `persistedAllowedPairs[handle][account] = true`. Bu izni geri alan bir fonksiyon yoktur. Zama'nın belgeleri de izinleri açıkça "kalıcı" olarak tanımlar [R4]. ACL'deki tek engelleme mekanizması olan `blockAccount` yalnızca sahibin (Zama yönetişimi) çağırabildiği bir fonksiyondur. Üstelik yalnızca `allow`, `allowForDecryption` ve `allowTransient` çağrılarını engeller; **şifre çözme yolunda hiç kontrol edilmez** [R2, R3].

Değerlendirme: İzinlerin kalıcı olması bilinçli bir tercihtir (uygulamaların ihtiyacı). Eksik olan, **kullanıcının kendi hesabını emekliye ayırabileceği** bir mekanizmadır.

B2. İptal mekanizması imzaya karşıdır, anahtara karşı değil

`invalidateDecryptionSignaturesBefore(ts)`, connector'da `startTimestamp < ts` koşuluyla uygulanır [R2, R3]. `startTimestamp` alanını imzayı atan kişi belirler. Dolayısıyla anahtarı elinde tutan biri yeni tarihli bir izin imzalar ve kontrolden geçer. Fonksiyon geleceğe dönük bir zaman damgasını da kabul etmez (`InvalidationTimestampInTheFuture`). Kullanıcı "bundan sonra hiçbir imzayı kabul etme" diyemez.

Değerlendirme: Sızan tek tek izinlere karşı doğru bir tasarım. Anahtar ele geçirildiğinde ise hiçbir koruma sağlamıyor. **Gözden kaçan durum.**

B3. Zincir üstü kapı: geçmiş tek işlemle aktarılabilir veya herkese açılabilir

`allow(handle, account)` yalnızca "çağıran bu handle'a erişebiliyor mu" diye bakar [R2]. Dolayısıyla bir EOA kendi geçmiş handle'larını herhangi bir adrese yeniden izin verebilir ya da `allowForDecryption` ile herkese açık hale getirebilir. Bu, B1 ile birleştiğinde şu anlama gelir: Anahtar el değiştirdiğinde, kullanıcı tarafında şifre çözme yetkisine yapılacak hiçbir değişiklik (örneğin imza doğrulamayı başka bir anahtara taşımak) geçmişi korumaz. Çünkü aynı anahtar zincir üstünden de yetki aktarabilir.

Değerlendirme: Bu, ilk tasarım fikrimizi (Bölüm 6, A1) geçersiz kılan bulgudur. **Gözden kaçan durum:** Kullanıcı yetkisinin yaşam döngüsü tasarlanmamış.

B4. Zincir dışı kapı: Ethereum'un acil durum planı FHE'yi korumaz

Vitalik Buterin'in kuantum acil durum önerisi, açık anahtarı görünmüş EOA'ların **işlemlerini** dondurmayı öngörür [R14]. Zama'da kullanıcı şifre çözmesi ise bir işlem değil, zincir dışında doğrulanan bir EIP-712 imzasıdır. Connector bu imzayı Rust içinde `ecrecover` ile doğrular [R5]. Zincir dondurulsa bile bu doğrulama çalışmaya devam eder.

Değerlendirme: **Gözden kaçan durum.** Ekosistemdeki kuantum geçiş planları "işlem" odaklıdır; zincir dışı imza doğrulayıcılarını hesaba katmaz.

B5. Önce `ecrecover`: 7702 ile yükseltilmiş hesaplarda eski anahtar geçerli kalır

`verify_signature`, `ecrecover` ile bulunan adres beyan edilen adresle eşleşirse imzayı **hiç ERC-1271'e sormadan** kabul eder [R5]. EIP-7702 ile kodu bir PQ akıllı cüzdana delege edilmiş bir EOA'nın orijinal ECDSA anahtarı, Zama şifre çözmesi için geçerliliğini korur.

Ethereum tarafında bu sorunu kapatmak için EIP-7851 (delege edilmiş EOA anahtarının devre dışı bırakılması) [R10] ve EIP-8151 (devre dışı bırakılmış anahtar için `ecrecover` sıfır döndürsün) [R11] önerilmiştir. EIP-7851 tartışmasında ArikG, ERC-1271'i önce kontrol etmeyen sözleşmelerin bu korumayı delebileceğini not etmiştir [R10]. Bu genel sınıf bilinen bir sorundur. **Bizim bulgumuz bunun FHE'ye özgü uzantısıdır:** Zama'nın doğrulaması EVM'deki precompile'ı değil, zincir dışındaki bir kütüphaneyi kullandığı için EIP-8151 kabul edilse bile Zama'ya otomatik olarak yansımaz.

Değerlendirme: ERC-1271 desteği bir Safe/multisig ihtiyacı için eklenmiş (RFC-012). Kuantum ve 7702 senaryosu düşünülmemiş. **Gözden kaçan durum**, düzeltmesi küçük.

B6. PQ akıllı hesaplar 250.000 gas tavanına takılır

ERC-1271 `isValidSignature` çağrısı varsayılan olarak 250.000 gas ile sınırlıdır [R6]. Bu bir DoS korumasıdır ve Safe gibi cüzdanlar için yeterlidir. Ancak Solidity'deki PQ imza doğrulayıcıları şöyle:

Doğrulayıcı	Gas (doğrulama başına)	Kaynak
ML-DSA-44 (Fireblocks)	~1,23M	[R12]
Falcon, NIST uyumlu (ZKNOX)	~3,9M	[R12]
ETHFALCON (Keccak varyantı)	~1,5M	[R12]
En iyi deneysel Falcon	~0,65M	[R12]
EIP-8051 ML-DSA precompile (taslak)	~4.500	[R12]

Bugün çalışan hiçbir PQ doğrulayıcı bu tavanın altına girmiyor. Dolayısıyla PQ bir akıllı hesap Zama'da şifre çözmeyi yetkilendiremez. Bunun Sepolia'da doğrulanması gerekiyor (E4).

Değerlendirme: Tavanın değeri bilinçli bir tercih (DoS). PQ hesapları dışlaması ise muhtemelen düşünülmemiş bir yan etki.

B7. KMS'in PQ imzaları uçtan uca akmıyor

KMS v0.15 sonuçları ML-DSA-44/65/87 ile de imzalayabiliyor [R7]. Ancak:

- connector KMS'ten yalnızca `Ecdsa256k1` istiyor [R8];
- relayer ve SDK 65 bayt olmayan her KMS imzasını reddediyor [R9];
- `KMSVerifier` yalnızca ECDSA doğruluyor [R2].

Düğümler arası TLS'i hibrit PQ'ya geçiren bir dal da açık (`chore/pq-tls-handshake`, 5 Ekim 2026).

Değerlendirme: **Bu Zama'nın aktif yol haritasıdır.** Altyapıdan yukarı doğru ilerliyorlar. Bu çalışmanın konusu değildir.

4.1 Bilinçli tercihler: dokunmadığımız şeyler

Davranış	Neden bilinçli	Önerimiz
İmzalı izin 365 güne kadar geçerli [R2]	Kullanıcı bir kez imzalasın, cüzdan sürekli açılmasın	Dokunmuyoruz
Delegasyon süresinde üst sınır yok [R2]	Akıllı hesap ve otomasyon kolaylığı	Dokunmuyoruz; mühürleme mevcut delegasyonları da kapatır (C1)
ERC-1271 gas tavanı	DoS koruması	Tavanı kaldırmıyoruz; PQ için ayrı bir yol öneriyoruz (C4)
ACL izinlerinin kalıcı olması	Uygulamaların ihtiyacı	Dokunmuyoruz; kullanıcıya kendi hesabını emekliye ayırma imkânı ekliyoruz

4.2 Fhenix CoFHE ile karşılaştırma

Fhenix'te `TaskManager.allow`, `allowGlobal` ve `allowForDecryption` herkes tarafından çağrılabilir; çağıranın izni varsa yetkiyi aktarır ya da handle'ı herkese açar [R15]. Kodda "allowances are never revoked" (izinler hiçbir zaman geri alınmaz) yorumu yer alıyor. Şifre çözme izinlerinde (ACP) iptal mekanizması isteğe bağlıdır ve iznin hangi iptal sözleşmesine bağlanacağına izni imzalayan karar verir [R16]. Anahtarı ele geçiren biri iptal edilemez bir izin imzalayabilir.

Dolayısıyla B1, B2 (farklı biçimde) ve B3 Fhenix'te de geçerlidir. Fhenix izinleri EIP-1271'i doğrudan destekler; gas tavanı (B6) ise Fhenix tarafında henüz doğrulanmadı.

5. Tasarım ilkeleri

1. **Yeni kavram icat etme, eksik parçayı ekle.** Anahtar değiştirme mekanizması Ethereum'da zaten var: hesap soyutlama.
2. **İki kapının ikisini birden kapat.** Zincir üstü (ACL) ve zincir dışı (connector) yolların ikisi de aynı karara bağlanmalı.
3. **Geri dönülmezlik.** Emekliye ayırma geri alınabilirse, anahtarı ele geçiren onu geri alır.
4. **Geriye dönük uyumluluk.** Hiçbir şey yapmayan kullanıcının deneyimi değişmemeli.
5. **Fail-closed.** Belirsizlik anında verinin sızması yerine erişimin geçici olarak kesilmesi tercih edilmeli.

6. Değerlendirilen ve reddedilen alternatifler

A1. Ayrı bir "Şifre Çözme Kimliği" kaydı (bizim ilk önerimiz). Hesaptan bağımsız, değiştirilebilir bir şifre çözme anahtarı kaydedilir; connector PQ imzayı kendi içinde doğrular. *Reddedilme nedeni:* B3. Anahtarı ele geçiren biri zincir üstünden `allow` ile geçmişi aktarır, kimlik değişikliği bunu engellemez. Ayrıca hesap soyutlamanın zaten sağladığı anahtar değiştirmeyi zincir dışında yeniden icat ediyordu.

A2. FHE anahtar rotasyonu / geçmişin yeniden şifrelenmesi. Erişim kontrolü sorununu çözmez; eski handle'lar yeni anahtar altında da aynı ACL kurallarıyla çözülür. Maliyeti de çok yüksektir.

A3. Süreli ACL izinleri. Bütün uygulamaların semantiğini değiştirir (ERC-7984 bakiyeleri). Geriye dönük uyumsuzdur.

A4. Hiçbir şey yapmamak, Ethereum'un acil durum çözümüne güvenmek. B4 ve B5 nedeniyle yetersiz: zincir dışı kapı açık kalır.

A5. Kullanıcıları yalnızca akıllı hesaba geçmeye yönlendirmek. Gerekli ama yeterli değil. Eski adres geçmişe erişimini korur (B1, B3), PQ akıllı hesaplar da tavana takılır (B6).

A6. KMS imzalarının PQ yapılması. Zama zaten yapıyor (B7). Kullanıcı yetkisini çözmez.

7. Önerilen tasarım: Emekli Et ve Taşı

Tasarım beş bileşenden oluşur. C1–C3 kullanıcının geçmişini korur, C4 PQ hesapları kullanılabilir kılar, C5 kuantum günü için protokol düzeyinde bir emniyet anahtarıdır.

C1. Kendini mühürleme (`sealAccount`)

Kullanıcının kendi hesabını şifre çözme açısından geri dönülmez biçimde emekliye ayırmasını sağlar.

```
// ACL'e eklenecekler (taslak)
mapping(address => bool) sealed;                      // ACLStorage içinde
event AccountSealed(address indexed account);

function sealAccount() external {
    ACLStorage storage $ = _getACLStorage();
    if ($.sealed[msg.sender]) revert AlreadySealed();
    $.sealed[msg.sender] = true;
    // Zincir dışı kapı: mevcut connector mantığını DEĞİŞTİRMEDEN kapatır.
    // Connector `startTimestamp < invalidatedBefore` ise reddediyor (B2);
    // sentinel değer her izni reddettirir.
    $.decryptionSignatureInvalidatedBefore[msg.sender] = type(uint256).max;
    emit AccountSealed(msg.sender);
}

// Zincir üstü kapı: mühürlü hesap yetki aktaramaz
modifier notSealed() { if (_getACLStorage().sealed[msg.sender]) revert
SenderSealed(); _; }
// allow, allowForDecryption, allowTransient, delegateForUserDecryption ->
notSealed

// Var olan delegasyonlar da kapanır
function isHandleDelegatedForUserDecryption(address delegator, ...) ... {
    if ($.sealed[delegator]) return false;
    ...
}
```

Tasarım kararları:

- **Sentinel değer:** `invalidatedBefore = 2^256-1`. Connector'da kod değişikliği gerektirmez; mevcut kontrol bütün izinleri zaten reddeder. Bunun için yalnızca `invalidateDecryptionSignaturesBefore` içindeki "gelecek tarih olamaz" kuralı, mühürleme yoluyla aşılır.
- **Geri dönülmezlik:** `unseal` fonksiyonu yoktur. Yönetişimin `unblockAccount` yetkisi mühürleme üzerinde etkili değildir.
- **Mühürlü adrese gelen yeni izinler:** Örneğin biri mühürlü adrese token gönderirse, sözleşme yine `allow` edebilir. Bu zararsızdır, çünkü şifre çözme yolu kapalıdır. Kullanıcıya "bu adres emekli" uyarısı SDK'nın işidir.

- **Delegasyonlar:** Connector'ın delege yolunda iptal kontrolü delegenin adresine bakar, delegasyonu verenin adresine bakmaz [R3]. Bu yüzden `isHandleDelegatedForUserDecryption` mühürlü delegasyon veren için `false` döndürmelidir.

C2. Toplu taşıma (`allowMany`)

Geçmişi yeni hesaba taşımak için, mühürlemeden **önce** eski hesap kendi handle'larını yeni hesaba izin verir. Bugün bu her handle için ayrı bir `allow` çağrısıdır. Toplu bir fonksiyon önerilir:

```
function allowMany(bytes32[] calldata handles, address account) external
notSealed whenNotPaused {
    for (uint256 i; i < handles.length; ++i) {
        if (!isAllowed(handles[i], msg.sender)) revert
SenderNotAllowed(msg.sender);
        $.persistedAllowedPairs[handles[i]][account] = true;
    }
    emit AllowedMany(msg.sender, account, handles);
}

// Tek işlemde "taşı ve mühürle"
function carryAndSeal(bytes32[] calldata handles, address newAccount) external;
```

- **Handle listesi:** SDK, `Allowed(*, account, handle)` olaylarından hesabın handle'larını çıkarır.
- **Tahmini maliyet:** Handle başına ~22–27K gas (yeni depolama slotu). 1.000 handle için ~25M gas eder; güncel L1 ücretleriyle birkaç dolar. **Bu bir tahmindir; E3'te ölçülecek.**
- **Tek işlemde birleştirme:** Taşıma ve mühürlemenin aynı işlemde yapılması, aradaki pencerede yarış koşulunu ortadan kaldırır.

C3. Hedef hesap kuralı ve 7702 düzeltmesi

- **Hedef hesap:** Geçmiş, ECDSA kökü olmayan saf bir akıllı hesaba (PQ imzalayıcılı) taşınmalıdır. EIP-7702 ile yükseltilmiş bir EOA, EIP-7851 ya da bir eşdeğeri devreye girene kadar ECDSA anahtarıyla kontrol edilmeye devam eder; bu yüzden **güvenli bir hedef değildir.**
- **Connector düzeltmesi:** `verify_signature`, `ecrecover` sonucunu kabul etmeden önce beyan edilen adresin kodunu okumalı ve EIP-8151 semantiğini zincir dışında uygulamalıdır. Kod boş ya da yalnızca 7702 delegasyon göstergesi değilse, ECDSA yolu reddedilir ve ERC-1271'e geçilir. Bu tek bir ek RPC okumasıdır; ileride EIP-7851 kabul edildiğinde 7702 hesaplarının da korunmasını sağlar.

C4. PQ hesaplar için doğrulama yolu

B6'yı çözmek için üç seçenek değerlendirildi:

Seçenek	Açıklama	Artı	Eksi
(a) Kod hash'i izin listesi	Bilinen PQ doğrulayıcı kodlarına (code hash) daha yüksek gas tavanı	Yalnızca yapılandırma değişikliği	İzin listesini yönetmek gerekir; connector başına eth_call maliyeti
(b) Şema ipuçlu yerel doğrulama	Hesap `pqAuthKey() → (scheme, pkHash)` sunar; connector imzayı Rust içinde doğrular (KMS'te ML-DSA kodu zaten var [R7])	Mikrosaniyeler; gas yok	Yeni arayüz; connector'a ML-DSA/Falcon/SLH-DSA kütüphanesi eklenir
(c) Precompile (EIP-8051/8052)	ML-DSA ~4.500 gas	Uzun vadede en temiz yol	Takvimi belirsiz

Öneri: Kısa vadede (a), çünkü en az değişikliği gerektiriyor. (b) ERC profilinde isteğe bağlı bir uzantı olarak tanımlanmalı. Uzun vadede (c) ya da EIP-8288 STARK yolu. Seçim E4 ve E6 ölçümlerine göre kesinleşecek.

C5. Fail-closed anahtarı (FHE için "bunker modu")

Protokol yönetişimi, kuantum günü ilan edildiğinde `ecdsaDecryptionAuthorityDisabled` bayrağını açar. Bayrak açıkken connector ECDSA yolundan gelen bütün kullanıcı şifre çözme izinlerini reddeder; yalnızca saf akıllı hesapların ERC-1271 / PQ yolu kabul edilir.

- Mühürlememiş ve geçiş yapmamış kullanıcı **erişimini geçici olarak kaybeder ama geçmişi sızmaz.**
- Geçiş yapmış kullanıcı hiçbir şey fark etmez.
- Bu karar, Ethereum'un EOA işlemlerini dondurmasıyla birlikte geçmişi iki kapıdan da korur (B3 ve B4).
- Geriye dönük uyumlu: bayrak varsayılan olarak kapalıdır.

7.1 Kullanıcı akışı

1. Kullanıcı bir PQ akıllı hesap oluşturur (Falcon/ML-DSA imzalayıcı, ECDSA kökü yok).
2. SDK eski adresin handle'larını listeler.
3. Eski adres `carryAndSeal(handles, newAccount)` çağırır: geçmiş yeni hesaba izinlenir, eski adres mühürlenir.
4. Varlıklar (ERC-7984 bakiyesi) normal transferle yeni hesaba taşınır.
5. Bundan sonra şifre çözme yalnızca yeni hesabın PQ imzasıyla yapılabilir (C4).

7.2 ERC-7984 PQ profili (taslak arayüz)

Profil yeni bir token tanımlamaz; ERC-7984 uygulayan sistemlerin destekleyeceği yetkilendirme davranışını tanımlar:

```
interface IConfidentialAuthorityLifecycle {
    function sealAccount() external;                                    // C1
    function isSealed(address account) external view returns (bool);
    function allowMany(bytes32[] calldata handles, address to) external;
// C2
    function carryAndSeal(bytes32[] calldata handles, address to) external;
// C1 + C2
    event AccountSealed(address indexed account);
}
// Zincir dışı doğrulayıcılar için kurallar (normatif metin):
//  R1: Mühürlü hesap için hiçbir izin kabul edilmez.
//  R2: ecrecover sonucu, beyan edilen adresin kodu boş ya da yalnızca 7702
göstergesi değilse kabul edilmez (EIP-8151 eşdeğeri).
//  R3: Fail-closed bayrağı açıkken ECDSA yolu reddedilir.
//  R4 (isteğe bağlı): pqAuthKey() sunan hesaplar için şema ipuçlu yerel
doğrulama.
```

8. Güvenlik analizi

Bulgu	Önerinin etkisi	Kalan risk
B1 Kalıcı hak	C1 ile kullanıcı kendi hesabını emekliye ayırır	Mühürlemeden önce çalınan anahtar
B2 İptal anahtara karşı değil	Sentinel değer bütün izinleri reddeder	Yok
B3 Zincir üstü aktarım	`notSealed` korumaları	Q-günü ile EOA dondurulması arasındaki pencere: mühürlememiş hesaplar için Ethereum düzeyinde çözüm gerekir
B4 Zincir dışı kapı	C1 (kullanıcı başına) + C5 (herkes için)	Yok
B5 7702 ve önce ecrecover	C3 connector düzeltmesi	EIP-7851 kabul edilene kadar 7702 hedef olarak güvensiz
B6 Gas tavanı	C4	(a) izin listesinin yönetimi
B7 KMS imzaları	Kapsam dışı	Zama'nın yol haritası

Klasik saldırgana (T1) karşı: Kullanıcı önceden saf akıllı hesaba geçmiş ve eski adresi mühürlemişse, imzalayıcı anahtarının çalınması akıllı hesabın anahtar değiştirmesiyle telafi edilir ve geçmiş korunur. Anahtar çalındıktan **sonra** mühürlemek ise yarışa dönüşür; tasarım bunu garanti etmez.

Kuantum saldırgana (T2) karşı: Q-gününden önce geçiş yapan kullanıcılar tamamen korunur. Diğerleri C5 sayesinde erişimini kaybeder ama verisi sızmaz.

9. Maliyet

İşlem	Tahmini maliyet	Durum
`sealAccount`	~45–50K gas (iki yeni slot + olay)	Tahmin, E6'da ölçülecek
`allowMany`, handle başına	~22–27K gas	Tahmin, E3 / E6
Connector'da ek kod okuması (C3)	İzin başına 1 RPC	Ölçülecek
PQ `isValidSignature`, (a) seçeneği	0,65–1,5M gas eth_call	E4
PQ yerel doğrulama, (b) seçeneği	Mikrosaniyeler	E6

10. Değerlendirme planı

Bütün deneyler **yalnızca kendi hesaplarımız ve kendi deploy ettiğimiz test sözleşmeleriyle** yapılacaktır. Başkalarına ait hiçbir veri okunmayacak, hiçbir üçüncü taraf hedef alınmayacaktır.

- **E1 – Sürüm doğrulaması:** Sepolia'daki ACL'in sürümünü ve fonksiyonlarını oku. Relayer'ın hangi yolu kullandığını belirle (yeni HTTP yolu mu, eski gateway yolu mu).
- **E2 – B2 doğrulaması:** Kendi test hesabımızla bir izin imzala, `invalidateDecryptionSignaturesBefore` çağır, ardından yeni tarihli bir izin imzala ve şifre çözmenin başarılı olduğunu göster.
- **E3 – B3 ve maliyet:** Kendi test ERC-7984 tokenımızda iki test hesabı arasında handle'ları `allow` ile aktar; handle başına gas'ı ölç.
- **E4 – B6 doğrulaması:** Sepolia'ya bir PQ akıllı hesap (Falcon ya da ML-DSA doğrulayıcılı) deploy et; ERC-1271 ile şifre çözme dene. Beklenen sonuç: tavan nedeniyle başarısızlık. `isValidSignature` gas'ını ölç.
- **E5 – B5 doğrulaması:** Kendi test EOA'mızı EIP-7702 ile bir akıllı cüzdana delege et; eski ECDSA anahtarıyla şifre çözmenin hâlâ çalıştığını göster.
- **E6 – Prototip:** fhevm test ortamının yerel kopyasında C1, C2, C3, C4(a/b) ve C5'i uygula. Uçtan uca göster:
  1. taşı ve mühürle;
  2. eski anahtarla gelen iznin reddedilmesi;
  3. eski anahtarla `allow` denemesinin geri çevrilmesi;
  4. PQ hesapla şifre çözme;
  5. fail-closed davranışı.

  Ayrıca bütün gas ve gecikmeleri ölç.
- **E7 – Fhenix:** Aynı bulguların Fhenix testnet'inde geçerliliğini kendi test hesaplarımızla doğrula.
- **E8 – Alternatiflerin karşılaştırması:** A1–A6 ve önerinin aynı tablo üzerinde maliyet ve kapsam karşılaştırması.

11. Sınırlamalar ve açık sorular

1. Bulgular `main` dalından; mainnet ve Sepolia farklı olabilir (E1).
2. Öneri Zama'nın host sözleşmelerinde ve connector'ında değişiklik gerektiriyor; tek başımıza mainnet'e uygulayamayız.
3. Geçmiş handle'ların karşı taraflardaki kopyaları (alıcının bakiyesi, sözleşmelerin izinleri) kullanıcının kontrolünde değildir.
4. `allowForDecryption` ile daha önce herkese açılmış handle'lar zaten herkese açıktır.
5. Zama, "anahtar = kimlik; bu tasarım gereği böyle" diyebilir. Kurumsal kullanım (maaş, hazine) için geçmiş gizliliği argümanı bu noktada belirleyicidir.
6. C4(a) izin listesinin kim tarafından ve nasıl yönetileceği açık bir sorudur.
7. Connector'ın tamamladığı ve sakladığı şifre çözme sonuçları süresiz sunulur [R3]. Bu sonuçlar talep sahibinin ML-KEM anahtarına şifreli olduğu için ek bir risk görmüyoruz, ama yine de incelenmeli.

12. İlgili çalışmalar

- **ERC-8231 (PQ Anahtar Kaydı):** Yaşam döngüsü olan genel bir PQ anahtar kaydı. Şifre çözme yetkisi, kilitleme ve ACL ilişkisini kapsamaz [R17]. C4(b)'de anahtar kaydı katmanı olarak kullanılabilir.
- **EIP-7851 ve EIP-8151:** EOA anahtarının devre dışı bırakılması ve bunun `ecrecover`'a yansıtılması [R10, R11]. Bu çalışma, zincir dışı doğrulayıcıların bu korumayı miras almadığını gösteriyor.
- **ArikG'nin EIP-7851 yorumu ve HPS taslağı:** ERC-1271'i önce kontrol etmeyen doğrulayıcıların korumayı deldiği gözlemi [R10]. Bu çalışma onun FHE şifre çözmesine özgü örneğidir.
- **Zama KMS v0.15 ML-DSA imzaları ve PQ TLS dalı:** Altyapı tarafındaki tamamlayıcı çalışmalar [R7].
- **Mahdi171, "gizli açık anahtar + ZK" önerisi (Nisan 2026):** EOA'ları PQ'ya yükseltme yaklaşımı. C4'ün gelecekteki STARK tabanlı bir alternatifi.

13. Sorumlu paylaşım

Bulgular bugün istismar edilebilir bir açık değildir: B2 ve B3 ancak anahtarı zaten ele geçirilmiş bir hesap için anlam taşır, B4–B6 ise kuantum senaryosu ve tasarım eksikliğidir. Yine de yayından önce Zama ve Fhenix ekipleriyle özel olarak paylaşılacak ve geri bildirimleri bu belgeye eklenecektir.

Ek A. Kod referansları (20 Ekim 2026, `main`)

Kod	Dosya ve satır	İçerik
R1	`zama-ai/kms` docs/guides/entry_points.md	gRPC şifre çözme ucunun kendi erişim kontrolü yok
R2	`zama-ai/fhevm` host-contracts/contracts/ACL.sol: 230 (`allow`), 248 (`allowForDecryption`), 273 (`invalidateDecryptionSignaturesBefore`), 326 (`delegateForUserDecryption`), 561 (`isHandleDelegatedForUserDecryption`), 601 (`blockAccount`); KMSVerifier.sol: 416 (`_recoverSigner`); gateway-contracts/contracts/Decryption.sol: 129 (365 gün sınırı)	ACL ve doğrulayıcılar
R3	`zama-ai/fhevm` kms-connector/crates/kms-worker/src/core/event_processor/decryption.rs (imza, iptal ve ACL kontrolleri; iptal kontrolü 522) ve kms-connector/docs/architecture.md	Connector kontrolleri
R4	`zama-ai/fhevm` docs/solidity-guides/acl/README.md	İzinlerin kalıcı olduğu
R5	`zama-ai/fhevm` shared/user-decryption-signature/src/lib.rs: 196	Önce ecrecover, sonra ERC-1271
R6	`zama-ai/fhevm` kms-connector/crates/kms-worker/src/core/config.rs: 274	250.000 gas tavanı
R7	`zama-ai/kms` core/service/src/cryptography/signing/mldsa.rs; `chore/pq-tls-handshake` dalı	ML-DSA imzaları, PQ TLS
R8	decryption.rs: 614	Yalnızca `Ecdsa256k1` isteniyor
R9	`zama-ai/fhevm` relayer-http/crates/relayer-http/src/kms_aggregator/flows/mod.rs: 51; sdk/js-sdk/src/core/base/sign.ts: 15	65 baytlık imza zorunluluğu
R10	ethereum-magicians.org/t/eip-7851-code-controlled-eoa-delegation/22344	EIP-7851, ArikG yorumu
R11	ethereum-magicians.org/t/eip-8151-ecdsa-authority-deactivation-aware-ecrecover/27690	EIP-8151
R12	Fireblocks evm-ml-dsa-verifier; ZKNOX ETHFALCON / ETHDILITHIUM; EIP-8051	PQ doğrulayıcı gas değerleri
R13	KuCoin haber (17 Ekim 2026), Fhenix “quantum-resistant by design”	Kamusal anlatı
R14	ethresear.ch/t/how-to-hard-fork-to-save-most-users-funds-in-a-quantum-emergency/18901	Ethereum acil durum planı
R15	`FhenixProtocol/cofhe-contracts` TaskManager.sol: 947–971; ACL.sol: 160–211	Fhenix izin aktarımı

Kod	Dosya ve satır	İçerik
R16	`FhenixProtocol/cofhe-contracts` Permissioned.sol (ACP, revokerContract)	Fhenix izin iptali
R17	ethereum-magicians.org/t/erc-8231-post-quantum-key-registry/28235	ERC-8231